

Shula AI Developer Portal

Table of contents

1. Shula AI Moodle Integration (local_shula)	3
1.1 Core Features	3
1.2 System Requirements	3
1.3 Installation & Setup	3
2. Architecture Overview	5
2.1 1. Asynchronous Event-Driven Flow	5
2.2 2. Key Architectural Resilience Safeguards	5
3. Services	7
3.1 Payload Builder Service	7
3.2 Secure Webhook API Client	9
4. Tasks & Webhooks	11
4.1 Event Observer	11
4.2 Asynchronous Background Tasks	13
5. Testing Guidelines	15
5.1 1. PHPUnit Unit Testing	15
5.2 2. Behat Acceptance Testing	15

1. Shula AI Moodle Integration (`local_shula`)

Welcome to the official developer and administrator documentation for the `local_shula` Moodle plugin.

`local_shula` serves as the real-time, asynchronous bridge ("Door A") between the Moodle Learning Management System (LMS) and the Shula AI platform. Its primary role is to proactively push course structure, section information, module availability, visibility toggles, and teacher-authored material references to the Shula Django backend.

1.1 Core Features

- **Asynchronous Decoupled Execution:** Avoids blocking the Moodle UI thread by offloading HTTP dispatches to Moodle's native Ad-hoc Task API.
 - **Secure Webhook Dispatch:** All outbound payloads are HMAC-SHA256 signed with replay protection and safe DNS/SSRF resolution using Moodle's built-in curl utility.
 - **Strict Student Privacy Guardrails:** Strict zero-PII leakage policy. The plugin never reads, caches, or transmits student names, grades, emails, or student-submitted content.
 - **Teacher-Authored Whitelist:** Only syncs folders and files uploaded by teachers, completely ignoring forums, wikis, glossaries, database submissions, and assignment uploads from students.
 - **AI Opt-Out Tag:** Teachers can attach a native Moodle tag (default `no-shula`) to any activity or section to set `ai_restricted = true` and exclude its files from ingestion.
 - **Restore Storm Protection:** Prevents spamming the Shula backend with event webhooks during course restore or plugin installation, seamlessly falling back to a single course-wide paginated bulk-sync task.
 - **Paginated Course Bulk Sync:** Syncs full course hierarchies in optimized chunks of 5 sections per HTTP request to ensure massive courses do not hit timeout or payload limits.
-

1.2 System Requirements

- **PHP:** 8.1 or higher.
 - **Moodle:** 4.1 to 4.5.
 - **Moodle PHPUnit Environment** (optional, for running tests).
 - **Python 3+** (optional, for running this MkDocs documentation site).
-

1.3 Installation & Setup

1. Copy or symlink the `local_shula` folder into your Moodle installation's `local/` directory:

```
ln -s /path/to/local_shula /path/to/moodle/local/shula
```

2. Run Moodle database upgrades via the admin interface or CLI:

```
php admin/cli/upgrade.php
```

3. Navigate to **Site administration > Plugins > Local plugins > Shula AI Integration** (`/local/shula/settings.php`) to configure:
4. **Institution ID:** Your unique Shula AI account identifier.
5. **LTI Tool Identifier:** The name or URL matching the Shula LTI activity in courses.
6. **Webhook Endpoint:** The Django receiver URL.
7. **Webhook Secret:** The secret key used for HMAC signature generation.

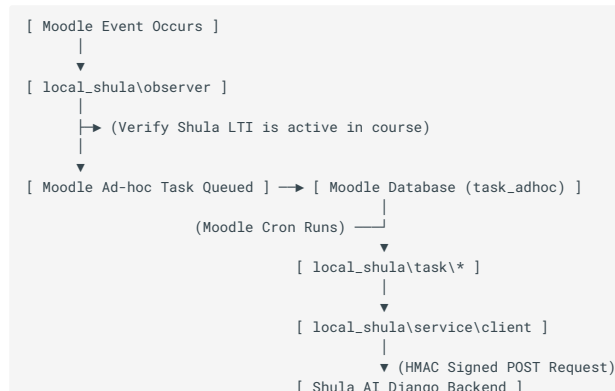
8. **Opt-Out Tag:** The tag used by teachers to restrict content (defaults to `no-shula`).

2. Architecture Overview

The `local_shula` plugin acts as a proactive, real-time, event-driven bridge ("Door A") between a Moodle LMS instance and the Shula AI Django backend. It ensures that the Shula AI platform is constantly synchronized with course structures, sections, and module modifications, without introducing performance overhead to Moodle's frontend interface.

2.1 1. Asynchronous Event-Driven Flow

To prevent slow HTTP requests from blocking the Moodle UI and degrading the teacher or student experience, `local_shula` employs a strictly asynchronous, decoupled architecture:



2.1.1 Flow Lifecycle

- User Action:** A teacher makes a change (e.g., adds a folder, hides a section, deletes a URL).
- Event Interception:** Moodle fires an event (e.g., `\core\event\course_module_created`), which is caught by `local_shula\observer`.
- LTI Validation:** The observer verifies if Shula is active in that course (by searching for configured Shula LTI activities in the course). If inactive, execution terminates instantly.
- Task Queueing:** An ad-hoc background task is queued with minimal required variables (such as `courseid` and `cmid`) using Moodle's native task manager:

```
\core\task\manager::queue_adhoc_task($task);
```

- Background Dispatch:** The Moodle Cron engine executes the task asynchronously, building the required secure JSON payload via `payload_builder` and dispatching it to Shula via `client::send()`.

2.2 2. Key Architectural Resilience Safeguards

2.2.1 2.1. Restore Storm Protection

During course restores, imports, or plugin installation, thousands of events (such as module and section creations) can occur simultaneously. If left unhandled, this would spawn thousands of HTTP requests, stalling Moodle's cron runner and crashing or rate-limiting the Shula backend.

To prevent this, `local_shula` implements **Restore Storm Protection**: - The event observer checks if Moodle is currently restoring a course or installing a plugin using `$CFG->restoringcourse` and `$CFG->installing`. - If a restore is active, normal module/section creation/update event hooks are immediately aborted. - On the `course_restored` event (triggered at the very end of a course restoration), a single `local_shula\task\bulk_sync` task is queued to synchronize the entire course in one structured, paginated transaction.

2.2.2 2.2. Task Debouncing

To prevent duplicate requests for the same event type occurring in quick succession, the helper methods in `\local_shula\observer` query Moodle's database to check if an identical ad-hoc task is already pending execution:

```
if ($DB->record_exists('task_adhoc', ['classname' => $classname, 'customdata' => $json_data])) {
    return; // Skip duplicate queueing
}
```

This guarantees that multiple rapid clicks or fast edits to the same course or module do not result in redundant back-to-back background jobs.

2.2.3 2.3. Request-Level Static Caching

Determining if Shula is active in a course requires evaluating LTI activity instances. Since Moodle's event subsystem may fire several observers during a single page load, the plugin caches the result of the LTI check in a static property (`self::$active_course_cache`):

```
if (isset(self::$active_course_cache[$courseid])) {
    return self::$active_course_cache[$courseid];
}
```

This completely avoids duplicate DB lookups for the LTI tool configuration during a single request lifecycle.

2.2.4 2.4. Snapshot Preservation on Deletions

When an item is deleted in Moodle, its database records are wiped **before** the background cron job has a chance to execute. To ensure the background client knows *what* was deleted, the event observer extracts a pre-deletion database snapshot (or pre-builds a payload snapshot) and encapsulates it directly into the ad-hoc task's custom data:

```
$course_item = \local_shula\service\payload_builder::build_course_item($course);
$task->set_custom_data([
    'event_type' => 'lti_tool_deleted',
    'courseid' => $event->courseid,
    'course_item' => $course_item
]);
```

This enables the background task to accurately reconstruct and dispatch deletion payloads even after Moodle has cleared the records from its database.

3. Services

3.1 Payload Builder Service

The `payload_builder` class (`local_shula\service\payload_builder`) is responsible for constructing secure, standardized, hierarchical JSON representations of Moodle courses, sections, and activities. This structure matches the input schemas expected by the Shula AI Django backend.

3.1.1 1. Structure Mapping (Schemas)

`payload_builder` builds three primary objects corresponding to the core components of a Moodle course structure:

1.1. Course Item (`MoodleCourseItem`)

Formed via `build_course_item($course)`. It returns: * **moodle_course_id (int)**: The unique course ID. * **fullname (string)**: Full name of the course. * **categoryid (int)**: Course category ID. * **is_visible (int)**: Visibility setting (0/1). * **summary (string)**: The course description. * **summaryformat (int)**: Format of the description. * **lang (string)**: The course language code (defaults to Moodle standard `$CFG->lang` or `'en'`). * **format (string)**: The layout format (e.g., `topics`, `weeks`). * **current_marker (int)**: Highlighting marker for the active topic/week section.

1.2. Section Item (`MoodleSectionItem`)

Formed via `build_section_item($section_info, $modinfo, $include_modules)`. It includes: * **moodle_section_id (int)**: The unique section database ID. * **section_number (int)**: The sequential section index (0, 1, 2, ...). * **name (string)**: Section title (custom or default). * **summary (string)**: HTML introduction of the section. * **is_visible (int)**: Visibility setting of the section (0/1). * **unlock_date (int|null)**: Extracted start restriction timestamp. * **availability_rule (array|null)**: The full availability JSON AST. * **modules (array)**: Array of nested Module items (only included when `$include_modules` is true).

1.3. Module Item

Formed via `build_module_item($cm)`. It combines database-specific fields, virtual HTML, and physical files: * **course_module_id (int)**: Moodle's unique Course Module ID (`cmid`). * **instance_id (int)**: The database ID of the specific activity (e.g., specific `page` ID, `url` ID). * **name (string)**: The activity name. * **modname (string)**: The module type name (e.g., `resource`, `page`, `folder`, `assign`, `url`). * **is_visible (int)**: Visibility toggle (0/1). * **unlock_date (int|null)**: Extracted availability timestamp. * **ai_restricted (bool)**: Set to true if an opt-out tag is detected. * **availability_rule (array|null)**: Detailed availability rule array. * **files_count (int)**: Combined count of attached files. * **files_size (int)**: Combined byte size of attached files. * **files (array)**: Consolidated content schema records (representing physical and virtual files).

3.1.2 2. Zero-PII Privacy Guardrails

Student data security is enforced directly at the payload construction layer to ensure that student records never reach the AI:

2.1. Teacher-Authored Filearea Allowlist

The plugin uses a strict, closed mapping representing allowed, secure teacher-authored file areas (`get_safe_teacher_fileareas()`):

```
public static function get_safe_teacher_fileareas(): array {
    return [
        'mod_resource' => ['content', 'intro'],
        'mod_page'      => ['content', 'intro'],
        'mod_folder'    => ['content', 'intro'],
        'mod_assign'    => ['introattachment', 'intro'],
        'mod_book'      => ['chapter', 'intro'],
        'mod_scorm'      => ['package', 'intro'],
        'mod_imscp'      => ['content', 'intro'],
        'mod_lesson'    => ['page_contents', 'intro'],
        'mod_url'        => ['intro'],
        'mod_forum'      => ['intro'],
        'mod_quiz'       => ['intro'],
        'mod_label'     => ['intro']
    ];
}
```

```
};
}
```

* **Student-contributed files** (such as forum posts, assignment submissions, wiki attachments, or database submissions) are **completely omitted** because their respective fileareas are not listed on the allowlist.

2.2. AI Opt-Out Tag (no-shula)

Teachers can exclude individual activities or complete sections from AI access by adding a configured opt-out tag (configured in settings, defaults to no-shula):

1. **Detection:** The builder queries Moodle's Core Tag API (\core_tag_tag::get_item_tags()) for the course module.
2. **Exclusion:** If the tag matches, ai_restricted is set to true and the files array is completely wiped before dispatch, preventing file uploads from reaching the AI tutor.

3.1.3 3. Advanced Availability Date Extraction

Moodle stores availability constraints (conditional release rules) in a nested, Boolean logical AST structure (e.g. { "op": "&", "c": [{ "type": "date", "d": ">=", "t": 1718064000 }] }).

To feed the AI tutor with accurate start times, the extract_unlock_date() helper dynamically resolves this structure using a **recursive AST walker**:

- It recursively parses nested rule objects.
- It filters by type date and direction >= or > (meaning "available from").
- In cases of multiple restrictive date rules (e.g. complex nested logical blocks), it keeps the **most restrictive (latest) date** as the ultimate unlock_date timestamp.

3.2 Secure Webhook API Client

The `client` class (`local_shula\service\client`) handles the outbound network transport layer of `local_shula`. It ensures that all JSON payloads are delivered to the Shula AI Django backend securely, utilizing cryptographic signatures, strict TLS validation, and replay protections.

3.2.1 1. Webhook Signature Generation

Every outbound HTTP POST request is signed with a cryptographic signature to verify authenticity and prevent tampering:

1.1. HMAC-SHA256 Algorithm

1. The JSON payload is constructed and packed:

```
$payload['issuer'] = $CFG->wwwroot;
$payload['sent_at'] = time(); // Replay protection timestamp
$json_payload = json_encode($payload);
```

2. A hash-based message authentication code (HMAC-SHA256) is calculated over the raw, unescaped JSON string using the shared `shula_webhook_secret` as the key:

```
$signature = hash_hmac('sha256', $json_payload, $secret);
```

3. The resulting signature is sent in the custom `X-Moodle-Signature` HTTP header.

1.2. HTTP Header Headers

- **Content-Type**: `application/json`
- **X-Signature-Version**: `v1` (Provides backward compatibility during updates)
- **X-Moodle-Signature**: The calculated HMAC-SHA256 signature string.

3.2.2 2. Secure Transport Parameters

The client utilizes Moodle's built-in `\curl` utility, which natively enforces internal safety checks (like preventing SSRF attacks against local loopback interfaces).

To ensure enterprise-grade network security, the client applies the following rigid curl options: * **Connection Timeout (CURLOPT_CONNECTTIMEOUT)**: `5 seconds` (Prevents blocking background threads if the Django backend is offline). * **Execution Timeout (CURLOPT_TIMEOUT)**: `60 seconds` (Provides sufficient time for large file-sync payloads to complete). * **TLS Peer Verification (CURLOPT_SSL_VERIFYPEER)**: `true` (Enforces TLS certificate chain validation). * **TLS Host Verification (CURLOPT_SSL_VERIFYHOST)**: `2` (Verifies that the target hostname matches the SSL certificate common name).

3.2.3 3. Developer Debug Logging

Logging full JSON bodies in production logs can clutter system diagnostics. `local_shula` implements **Dynamic Level Logging**:

- **Standard Mode**: Outputs a lightweight single-line trace to Moodle's `mtrace` log:

```
Shula Webhook: Dispatching 'file_created' event to Django at https://api.shula.ai/webhook/...
```

- **Developer Debug Mode**: When Moodle's Developer Debugging option is active (`DEBUG_DEVELOPER`), the client pretty-prints the complete formatted JSON schema payload directly to `mtrace` logs. This allows developers to easily inspect payload boundaries and file metadata structures during testing or local development.

3.2.4 4. Response & Retries Handling

Upon receiving the response, the client checks the HTTP status code: * **HTTP 2xx**: Considered a success. * **HTTP 4xx (Client Errors)**: Treated as a fatal/unrecoverable schema error. The calling task captures this exception, logs the event details, and discards the task to prevent infinite retry loops in Moodle's background queue. * **HTTP 5xx / Connection Timeout**: Treated as temporary server-side issues. The client throws a `\moodle_exception` which causes Moodle's task manager to reschedule and retry the dispatch later with exponential backoff.

4. Tasks & Webhooks

4.1 Event Observer

The `observer` class (`local_shula\observer`) is the entry point for real-time synchronization. It listens directly to Moodle's core event engine, filters out non-relevant changes, prevents restore storms, and triggers the asynchronous ad-hoc tasks that send sync payloads to Shula.

4.1.1 1. Event Registration (`db/events.php`)

All Moodle core events captured by the plugin are mapped inside `local_shula/db/events.php`:

Event Class	Observer Callback	Description
<code>\core\event\course_updated</code>	<code>course_updated</code>	Syncs course settings (fullname, visibility).
<code>\core\event\course_deleted</code>	<code>course_deleted</code>	Triggers immediate course-removal flow on Shula.
<code>\core\event\course_module_created</code>	<code>course_module_created</code>	Detects new content modules or the Shula LTI tool.
<code>\core\event\course_module_updated</code>	<code>course_module_updated</code>	Detects updates, name changes, or "Hide/Show" eyeball state changes.
<code>\core\event\course_module_deleted</code>	<code>course_module_deleted</code>	Tracks file deletions or LTI removal.
<code>\core\event\course_section_created</code>	<code>course_section_created</code>	Synchronizes newly created topic/week sections.
<code>\core\event\course_section_updated</code>	<code>course_section_updated</code>	Synchronizes section name edits, visibility, or dates.
<code>\core\event\course_section_deleted</code>	<code>course_section_deleted</code>	Notifies backend of section deletion.
<code>\core\event\course_restored</code>	<code>course_restored</code>	Triggers single-transaction full-course bulk synchronization.

4.1.2 2. Core Guard Methods

The observer performs several validation checks before queueing any background tasks:

2.1. LTI Activation Verification (`is_shula_active_in_course`)

The plugin only tracks courses where the **Shula LTI external tool** is active. - **Scanning Logic:** The observer queries Moodle for LTI instances in the course whose URL, secure URL, or tool config values contain the configured `shula_lti_identifier`. - **Database Safety:** When looking up matching strings, wildcards (such as `%` or `_`) are securely escaped using `$DB->sql_like_escape()` to prevent SQL injection or search pattern manipulation:

```
$safe_identifier = $DB->sql_like_escape($identifier);
$search = '%' . $safe_identifier . '%';
```

- **Static Request Cache:** Lookups are cached in a static array (`$active_course_cache`) for the duration of the current page request, avoiding redundant database queries when multiple events are fired.

2.2. Restore Check (`is_restoring`)

Checks Moodle's global state to see if a course is currently being restored, imported, or if the plugin is being installed:

```
private static function is_restoring() {
    global $CFG;
    return (!empty($CFG->restoringcourse) || !empty($CFG->installing));
}
```

If this method returns `true`, the observer aborts normal event processing to prevent "Restore Storms".

4.1.3 3. Advanced Deletion Handling (Snapshots)

When Moodle fires deletion events, the records are wiped from the database **before** the cron queue can process the ad-hoc task.

To solve this, the observer captures a **pre-deletion snapshot** inside the event thread and packages it in the task payload:

```
// Course deletion snapshot logic inside observer:
$course = $event->get_record_snapshot('course', $event->courseid);
$course_item = \local_shula\service\payload_builder::build_course_item($course);

$task = new \local_shula\task\send_webhook();
$task->set_custom_data([
    'event_type' => 'lti_tool_deleted',
    'courseid'   => $event->courseid,
    'course_item' => $course_item // Database snapshot passed in task!
]);
```

This enables the background task to build a complete course payload structure even after the DB record is gone.

4.2 Asynchronous Background Tasks

`local_shula` delegates HTTP webhooks to Moodle's native **Ad-hoc Task API**. This offloads outbound network operations from the user's interactive session to the background Moodle Cron scheduler, preserving frontend responsiveness.

All task classes are located under `local_shula\task` namespace and inherit from Moodle's core class `\core\task\adhoc_task`.

4.2.1 1. Paginated Course Bulk Sync Task (`bulk_sync`)

The `bulk_sync` task handles full course synchronizations. These typically occur when Shula is first activated in a course, or when a course is restored from a backup.

Pagination Mechanism

For medium to large courses, the payload representing all sections, modules, and file metadata can be massive. This risks PHP memory exhaustions, server timeout failures, or exceeding Django payload size limits.

To prevent this, `bulk_sync` segments sections into chunks: * **Chunk Size**: 5 sections per HTTP request. * **Payload Metadata**: The payload includes indexing properties so the Django backend can identify and assemble the chunks sequentially: - `chunk_index` (**int**): The current chunk sequence number (1-indexed). - `total_chunks` (**int**): The total number of chunks being dispatched.

```
// Section chunking logic in bulk_sync::execute()
$all_sections = $modinfo->get_section_info_all();
$chunk_size = 5;
$section_chunks = array_chunk($all_sections, $chunk_size);

foreach ($section_chunks as $chunk_index => $section_chunk) {
    // ... Build sections payload and send ...
}
```

4.2.2 2. Lightweight Section Sync Task (`send_section_webhook`)

The `send_section_webhook` task manages individual section creations, name updates, visibility toggles, or release-date restrictions.

- **Section Deleted Early Exit**: When a section is deleted, Moodle deletes its record from the database before the task runs. If the action is `section_deleted`, the task exits database lookup routines early and dispatches a lightweight payload containing only the `section_id` to notify Shula to wipe the section cache.
- **Section Created/Updated**: Fetches the section info from Moodle's `get_fast_modinfo()` and builds a complete representation of the section, including all nested modules.

4.2.3 3. General Event Sync Task (`send_webhook`)

The `send_webhook` task manages module-level events (`file_created`, `file_updated`, `file_deleted`) and specific course-wide events (`lti_tool_deleted`, `course_sync`).

- **Database Preservation**: For module-level creations/updates, it relies on Moodle's fast modinfo database caching to assemble the `module` item and its attached files.
- **Snapshot Integration**: For deletions, it automatically checks if a pre-built course snapshot was provided in custom data, using it instead of trying to read from a deleted database entry.

4.2.4 4. Queue Stall Prevention

If a webhook payload is structurally malformed or causes a schema validation error on the Django backend (HTTP 400 - 499), retrying it will always fail. In standard Moodle setups, throwing exceptions causes Moodle's task manager to retry the task indefinitely, stalling the background cron queue.

`local_shula` tasks resolve this by catching `webhook_failed` exceptions and checking the HTTP status code: - **HTTP 4xx (Client Error)**: The task intercepts the exception, logs a warning to the cron trace (`mtrace`), and exits gracefully. This automatically and safely removes the failing task from Moodle's queue, preventing queue congestion. - **HTTP 5xx (Server Error) or Connection Timeout**: The task re-throws the exception, triggering Moodle's native retry system with exponential backoff.

```
try {
    \local_shula\service\client::send($payload);
} catch (\moodle_exception $e) {
    if ($e->errorcode === 'webhook_failed' && is_numeric($e->a) && $e->a >= 400 && $e->a < 500) {
        mtrace("Shula: Bad request payload (HTTP {$e->a}). Dropping task to prevent queue stall.");
        return; // Graceful exit removes task from queue
    }
    throw $e; // Re-throw triggers cron retry
}
```

5. Testing Guidelines

The `local_shula` plugin prioritizes reliability and security. It includes both high-coverage PHPUnit tests and Behat acceptance tests to guarantee security lockouts, privacy compliance, and proper background execution.

5.1 1. PHPUnit Unit Testing

The PHPUnit suite validates class methods, caches, and security guardrails.

5.1.1 1.1. Setup & Execution

To run unit tests, ensure your Moodle development environment has PHPUnit configured:

```
# 1. Initialize PHPUnit environment (run from Moodle web root directory)
php admin/tool/phpunit/cli/init.php

# 2. Run the local_shula test group
vendor/bin/phpunit --group local_shula
```

5.1.2 1.2. Key Test Cases

API Client Tests (`tests/client_test.php`)

- `test_send_aborts_without_secret` : Confirms that the client aborts sending and returns `false` if the webhook secret is missing from settings.
- `test_send_throws_on_http_400` : Verifies that a `\moodle_exception` is thrown when the Shula endpoint returns a client error (HTTP 400), ensuring the calling task handles bad payloads gracefully.

Event Observer Tests (`tests/observer_test.php`)

- `test_is_restoring_returns_false_normally` : Asserts that under normal runtime conditions, the restore check is inactive.
- `test_active_check_caches_result` : Proves that checking LTI activation inside a course caches the result in memory, preventing duplicate database lookups during single page life cycles.
- `test_like_escape_neutralizes_wildcards` : Validates that if an administrator sets a wild wildcard (like `%`) as the LTI tool identifier, it is securely escaped and does not match random courses, mitigating database search injection vectors.

Payload Builder & Privacy Tests (`tests/payload_builder_test.php`)

This is the security gatekeeper suite. It verifies that **no student PII** can ever leak: * `test_high_risk_modules_are_secured` : Ensures that for modules like Forum and Assignment, only safe teacher-created fields (such as `introattachment`) are allowed, while student-contributed file areas (like `forum post`, `forum attachment`, `assignment submission_files`, or `feedback`) are strictly excluded. * `test_student_driven_modules_are_excluded` : Asserts that student-driven modules (such as Wikis, Glossaries, Workshops, and Databases) are entirely absent from the file-mapping configuration, preventing unintended student file leaks. * `test_allowlist_size_is_locked` : Locks the total size of the teacher filearea allowlist to exactly `12` components. If a developer attempts to add a new module support, they must intentionally modify this test, forcing a conscious security/privacy audit. * `test_no_shula_tag_excludes_files` : Validates that applying the `no-shula` tag to an activity successfully triggers `ai_restricted = true` and wipes its files from the generated JSON payload. * `test_extract_unlock_date_handles_nested_rules` : Ensures that recursive parsing of Moodle availability constraints correctly walks nested rule trees and extracts the correct Unix timestamp.

5.2 2. Behat Acceptance Testing

Behat tests verify end-to-end integration flows (e.g., verifying that making an edit in the Moodle UI correctly triggers and queues an ad-hoc webhook task).

5.2.1 Setup & Execution

```
# 1. Initialize Behat environment (run from Moodle web root directory)
php admin/tool/behat/cli/init.php

# 2. Run Behat scenarios tagged for local_shula
vendor/bin/behat --tags @local_shula
```

Our Behat feature (`tests/behat/webhook_dispatch.feature`) tests end-to-end UI actions, ensuring observers fire and tasks are scheduled cleanly in the background.